



Islamic University of Technology

Department of Computer Science and Engineering (CSE)

CSE 4205: Digital Logic Design
Summer, 2024-2025

Aashnan Rahman, Jr. Lecturer, CSE

Lecture 2

Binary Arithmetic and Complements

Contents

1	Radix and Diminished Radix Complements	2
1.1	Diminished Radix Complement ($r - 1$)'s Complement	2
1.2	Radix Complement r 's Complement	2
1.3	Binary Complements Summary	2
1.4	Examples	3
1.4.1	Base 10 (Decimal)	3
1.4.2	Base 2 (Binary)	3
1.4.3	Base 7	3
1.4.4	Base 5	3
2	Subtraction Using Complements	4
2.1	Example 1: Decimal (Base 10), 5-digit register	4
2.1.1	9's Complement Method	4
2.1.2	10's Complement Method	5
2.2	Example 2: Base 8 (4-digit register, 7^4)	6
2.2.1	7's Complement Method	6
2.2.2	8's Complement Method	7
3	BCD Arithmetic	8
3.1	Why Do We Add 0110 in BCD?	8
3.2	Rule for BCD Addition	8
3.3	Examples	9
3.3.1	Example 1: $257_{10} + 489_{10}$ in BCD	9
3.3.2	Example 2: $794_{10} + 428_{10}$ in BCD	9
4	Parity	10
4.1	How Even Parity Works (Sender Side)	10
4.2	Receiver Side: Error Detection	11
4.3	Limitation: When Parity Fails	11
5	Checksum	13
5.1	Example of Checksum (Base 10)	13
5.2	Checksum: Detection and Failure Cases	14
5.2.1	Case 1: Error Detected (Single Change)	14
5.2.2	Case 2: Error NOT Detected (Compensating Changes)	14

1 Radix and Diminished Radix Complements

Let a number N be represented in base r with n digits:

$$N = d_{n-1}d_{n-2} \dots d_0$$

1.1 Diminished Radix Complement $(r - 1)$'s Complement

The $(r - 1)$'s complement of N is obtained by subtracting each digit from $(r - 1)$:

$$\overline{N}^{(r-1)} = (r - 1 - d_{n-1})(r - 1 - d_{n-2}) \dots (r - 1 - d_0)$$

In practice, the resulting digits are written directly as a number.

1.2 Radix Complement r 's Complement

The r 's complement of N can be obtained by either of the following methods:

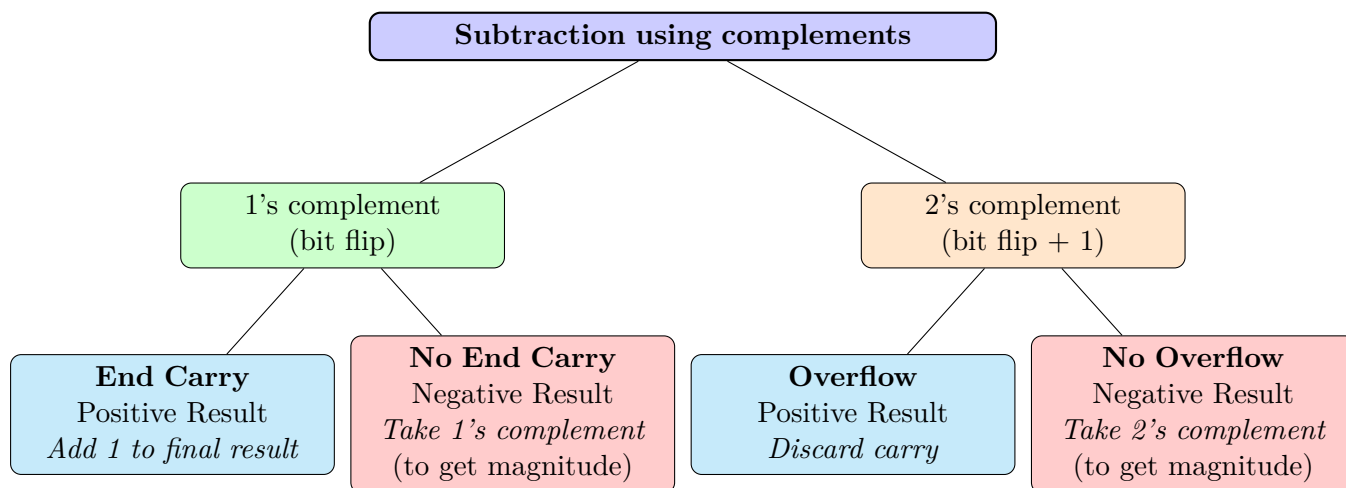
1. Addition Method:

$$\overline{N}^{(r)} = \overline{N}^{(r-1)} + 1$$

2. Direct Method:

Starting from the least significant digit (rightmost), retain all digits up to and including the first non-zero digit unchanged. Replace all more significant digits (to the left) by their $(r - 1)$'s complements.

1.3 Binary Complements Summary



1.4 Examples

1.4.1 Base 10 (Decimal)

Let $N = 5723_{10}$.

9's complement

$$5723_{10} \xrightarrow{\text{subtract each digit from 9}} 4276_{10}$$

10's complement

$$4276_{10} + 1_{10} = 4277_{10}$$

1.4.2 Base 2 (Binary)

Let $N = 10110_2$.

1's complement

$$10110_2 \xrightarrow{\text{subtract each digit from 1}} 01001_2$$

2's complement

$$01001_2 + 1_2 = 01010_2$$

1.4.3 Base 7

Let $N = 356_7$.

6's complement

$$356_7 \rightarrow 310_7$$

7's complement

$$310_7 + 1_7 = 311_7$$

1.4.4 Base 5

Let $N = 243_5$.

4's complement

$$243_5 \rightarrow 201_5$$

5's complement

$$201_5 + 1_5 = 202_5$$

2 Subtraction Using Complements

2.1 Example 1: Decimal (Base 10), 5-digit register

2.1.1 9's Complement Method

Given: $(5723)_{10} - (2486)_{10}$

Case 1: End Carry (Positive Result)

9's complement of $(02486)_{10} = (97513)_{10}$

$$\begin{array}{r} 0 \ 5 \ 7 \ 2 \ 3 \\ + \ 9 \ 7 \ 5 \ 1 \ 3 \\ \hline 1 \ 0 \ 3 \ 2 \ 3 \ 6 \end{array}$$

Observation: End carry occurs $\Rightarrow$ result is positive. Discard End Carry.

$$(03236)_{10} + (1)_{10} = (03237)_{10} \Rightarrow +(03237)_{10}$$

Given: $(2486)_{10} - (5723)_{10}$

Case 2: No End Carry (Negative Result)

9's complement of $(05723)_{10} = (94276)_{10}$

$$\begin{array}{r} 0 \ 2 \ 4 \ 8 \ 6 \\ + \ 9 \ 4 \ 2 \ 7 \ 6 \\ \hline 9 \ 6 \ 7 \ 6 \ 2 \end{array}$$

Observation: No end carry $\Rightarrow$ result is negative.

$$9's \text{ complement of } (96762)_{10} = -(03237)_{10}$$

2.1.2 10's Complement Method

Given: $(5723)_{10} - (2486)_{10}$

Case 1: End Carry (Positive Result)

10's complement of $(02486)_{10} = (97514)_{10}$

$$\begin{array}{r} 0 \ 5 \ 7 \ 2 \ 3 \\ + \ 9 \ 7 \ 5 \ 1 \ 4 \\ \hline 1 \ 0 \ 3 \ 2 \ 3 \ 7 \end{array}$$

Observation: End carry occurs $\Rightarrow$ discard carry.

$\Rightarrow +(03237)_{10}$

Given: $(2486)_{10} - (5723)_{10}$

Case 2: No End Carry (Negative Result)

10's complement of $(05723)_{10} = (94277)_{10}$

$$\begin{array}{r} 0 \ 2 \ 4 \ 8 \ 6 \\ + \ 9 \ 4 \ 2 \ 7 \ 7 \\ \hline 9 \ 6 \ 7 \ 6 \ 3 \end{array}$$

Observation: No end carry $\Rightarrow$ result is negative.

10's complement of $(96763)_{10} = -(03237)_{10}$

2.2 Example 2: Base 8 (4-digit register, 7^4)

2.2.1 7's Complement Method

Given: $(0524)_8 - (0137)_8$

Case 1: End Carry (Positive Result)

7's complement of $(0137)_8 = (7640)_8$

$$\begin{array}{r} 0 \ 5 \ 2 \ 4 \\ + \ 7 \ 6 \ 4 \ 0 \\ \hline 1 \ 0 \ 3 \ 6 \ 4 \end{array}$$

Observation: End carry occurs $\Rightarrow$ result is positive.

$$(0364)_8 + (1)_8 = (0365)_8 \Rightarrow +(0365)_8$$

Given: $(0137)_8 - (0524)_8$

Case 2: No End Carry (Negative Result)

7's complement of $(0524)_8 = (7253)_8$

$$\begin{array}{r} 0 \ 1 \ 3 \ 7 \\ + \ 7 \ 2 \ 5 \ 3 \\ \hline 7 \ 4 \ 1 \ 2 \end{array}$$

Observation: No end carry $\Rightarrow$ result is negative.

$$7's \text{ complement of } (7412)_8 = -(0365)_8$$

2.2.2 8's Complement Method

Given: $(0524)_8 - (0137)_8$

Case 1: End Carry (Positive Result)

8's complement of $(0137)_8 = (7641)_8$

$$\begin{array}{r} 0 \ 5 \ 2 \ 4 \\ + \ 7 \ 6 \ 4 \ 1 \\ \hline 1 \ 0 \ 3 \ 6 \ 5 \end{array}$$

Observation: End carry occurs $\Rightarrow$ discard carry.

$\Rightarrow +(0365)_8$

Given: $(0137)_8 - (0524)_8$

Case 2: No End Carry (Negative Result)

8's complement of $(0524)_8 = (7254)_8$

$$\begin{array}{r} 0 \ 1 \ 3 \ 7 \\ + \ 7 \ 2 \ 5 \ 4 \\ \hline 7 \ 4 \ 1 \ 3 \end{array}$$

Observation: No end carry $\Rightarrow$ result is negative.

8's complement of $(7413)_8 = -(0365)_8$

3 BCD Arithmetic

Binary Coded Decimal (BCD) is a number representation in which each decimal digit is represented separately using a 4-bit binary group.

For example:

$$(59)_{10} = (0101\ 1001)_{BCD}$$

Each decimal digit must remain in the range:

$$0000 \text{ to } 1001 \text{ (0 to 9)}$$

3.1 Why Do We Add 0110 in BCD?

In BCD addition, if a 4-bit group exceeds 1001_2 (9 in decimal), it becomes invalid BCD.

Valid BCD range:

$$0000_2 \rightarrow 1001_2$$

Invalid BCD range:

$$1010_2 \rightarrow 1111_2$$

When a result exceeds 9, we add:

$$0110_2 (= 6_{10})$$

Reason:

Adding 0110 corrects invalid binary results by:

- forcing a carry to the next decimal digit
- skipping invalid BCD codes (1010–1111)

This works because BCD is base-10 encoded in base-2 groups, and binary addition naturally produces excess values.

3.2 Rule for BCD Addition

After adding two BCD digits:

- If $\text{sum} \leq 1001$, it is valid $\rightarrow$ no correction needed
- If $\text{sum} \geq 1010$, add 0110 and carry propagate

3.3 Examples

3.3.1 Example 1: $257_{10} + 489_{10}$ in BCD

$$257_{10} = 0010\ 0101\ 0111_{BCD}$$

$$489_{10} = 0100\ 1000\ 1001_{BCD}$$

	0	0	1	0		0	1	0	1		0	1	1	1
+	0	1	0	0		1	0	0	0		1	0	0	1
	0	1	1	0 ⁺¹		1	1	0	1 ⁺¹		0	0	0	0
+						0	1	1	0		0	1	1	0
	0	1	1	1		0	1	0	0		0	1	1	0

Final Answer:

$$0111\ 0100\ 0110_{BCD} = 746_{10}$$

3.3.2 Example 2: $794_{10} + 428_{10}$ in BCD

$$794_{10} = 0111\ 1001\ 0100_{BCD}$$

$$428_{10} = 0100\ 0010\ 1000_{BCD}$$

	0	1	1	1		1	0	0	1		0	1	0	0
+	0	1	0	0		0	0	1	0		1	0	0	0
	1	0	1	1 ⁺¹		1	0	1	1 ⁺¹		1	1	0	0
+	0	1	1	0		0	1	1	0		0	1	1	0
	1	0	0	1		0	0	1	0		0	0	1	0

Final Answer:

$$1\ 0010\ 0010\ 0010_{BCD} = 1222_{10}$$

4 Parity

Parity is a simple error-detection method used in digital communication and storage. A **parity bit** is added to a block of data to make the total number of 1s either even (even parity) or odd (odd parity).

4.1 How Even Parity Works (Sender Side)

Given an n -bit data word, the sender counts the number of 1s. If the count is already even, the parity bit is set to 0; if odd, the parity bit is set to 1. So,

$$\text{Parity bit, } P = \begin{cases} 0, & \text{if the number of 1s in the data word is even} \\ 1, & \text{if the number of 1s in the data word is odd} \end{cases}$$

This ensures that the total number of 1s (data + parity) is **even**.

The even parity bit is simply the XOR (exclusive OR) of all data bits:

$$P = d_1 \oplus d_2 \oplus \cdots \oplus d_n$$

because XOR outputs 1 when the number of 1s in the inputs is odd. Thus, $P = 1$ makes the total number of 1s even.

In the receiver, the XOR of all received bits (including parity) should be 0 for no detected error:

$$d_1 \oplus d_2 \oplus \cdots \oplus d_n \oplus P = 0$$

If the result is 1, an error is detected. **But if the result is 0, it does not guarantee that there are no errors**

Example: Data = 1011001₂

1. Count the 1s in the data: $1 + 0 + 1 + 1 + 0 + 0 + 1 = 4$. The number of 1s is **even**.
2. Therefore, the even parity bit is **0**. (This can also be calculated using XOR: $P = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 \oplus 0 \oplus 1 = 0$.)
3. The transmitted codeword becomes:

Data bits: 1 0 1 1 0 0 1 Parity bit: **0**

Total bits sent: 1 0 1 1 0 0 1 **0**

(The parity bit is appended at the end for clarity.)

4.2 Receiver Side: Error Detection

The receiver extracts the data bits, counts the number of 1s (including the received parity bit). If the total count is **even**, the receiver assumes no error. If the total count is **odd**, an error has been detected.

Example: Correct Reception

The receiver gets:

10110010

Now XOR all bits:

$$1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 \oplus 0 \oplus 1 \oplus 0 = 0$$

Since the final XOR result is 0, the number of 1s is even.

Even parity $\Rightarrow$ No error detected

Example: Error Detected

Suppose a single bit flips during transmission. The third bit changes from:

$$10110010 \Rightarrow 10010010$$

The receiver gets:

10010010

Now XOR all bits:

$$1 \oplus 0 \oplus 0 \oplus 1 \oplus 0 \oplus 0 \oplus 1 \oplus 0 = 1$$

Since the final XOR result is 1, the number of 1s is odd.

Odd parity $\Rightarrow$ Error detected

4.3 Limitation: When Parity Fails

Parity can detect only an **odd number** of bit errors. If an even number of bits change, the parity still appears correct.

Case 1: Two Data Bits Flip (Undetected)

Original codeword:

10110010

Suppose two bits change during transmission:

$$10110010 \Rightarrow 11100010$$

Received:

11100010

XOR of all bits:

$$1 \oplus 1 \oplus 1 \oplus 0 \oplus 0 \oplus 0 \oplus 1 \oplus 0 = 0$$

Even parity $\Rightarrow$ Receiver incorrectly assumes no error

Case 2: Any Even Number of Bits Change

Example with four flipped bits:

$$10110010 \Rightarrow 00001000$$

XOR of all bits:

$$0 \oplus 0 \oplus 0 \oplus 0 \oplus 1 \oplus 0 \oplus 0 \oplus 0 = 0$$

Still even parity $\Rightarrow$ Error remains undetected

Case 3: Parity Bit Itself Changes

Original codeword:

$$10110010$$

Suppose only the parity bit flips:

$$10110010 \Rightarrow 10110011$$

Received:

$$10110011$$

XOR of all bits:

$$1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 \oplus 0 \oplus 1 \oplus 1 = 1$$

Odd parity $\Rightarrow$ Error detected

Parity detects only odd numbers of errors.

5 Checksum

A **checksum** is an error-detection method used in data communication. The sender divides data into fixed-size numbers, adds them, and sends the sum (checksum) along with the data.

At the receiver side, all received numbers (including checksum) are added again:

- If the final result is 0, the data is considered correct.
- If the result is non-zero, an error is detected.

5.1 Example of Checksum (Base 10)

Suppose the sender wants to transmit the following decimal numbers:

$$12, \quad 25, \quad 37, \quad 16$$

Step 1: Sender Adds the Data

$$12 + 25 = 37$$

$$37 + 37 = 74$$

$$74 + 16 = 90$$

Step 2: Generate the Checksum

We take the **10's complement** idea:

$$100 - 90 = 10$$

So the checksum is 10.

The sender transmits:

$$12, \quad 25, \quad 37, \quad 16, \quad 10$$

Step 3: Receiver Verification

The receiver adds all received values:

$$12 + 25 + 37 + 16 + 10 = 100$$

Now take discard overflow, so sum = 00

Result = 0 $\Rightarrow$ No error detected

If the final result is 0, the transmission is assumed correct.

5.2 Checksum: Detection and Failure Cases

5.2.1 Case 1: Error Detected (Single Change)

Original data:

12, 25, 37, 16

Sender sum:

$$12 + 25 + 37 + 16 = 90$$

Checksum (10's complement of 90):

$$100 - 90 = 10$$

Sender transmits:

12, 25, 37, 16, 10

Now suppose an error occurs during transmission:

$$37 \rightarrow 39$$

Received data:

12, 25, 39, 16, 10

Receiver sum:

$$12 + 25 + 39 + 16 + 10 = 102$$

Now discard the carry (10's complement step):

$$102 \Rightarrow 02$$

Now take 10's complement:

$$100 - 02 = 98$$

$$98 \neq 0 \Rightarrow \text{Error detected}$$

5.2.2 Case 2: Error NOT Detected (Compensating Changes)

Original transmitted data:

12, 25, 37, 16, 10

Suppose two errors occur (net change is zero):

$$25 \rightarrow 26, \quad 37 \rightarrow 36$$

Received data:

12, 26, 36, 16, 10

Receiver sum:

$$12 + 26 + 36 + 16 + 10 = 100$$

Discard carry:

$$100 \Rightarrow 00$$

10's complement:

$$100 - 00 = 100 \Rightarrow 00$$

0 $\Rightarrow$ No error detected

Checksum detects most errors, but compensating multi-bit errors may go undetected.
--